

Sofia Lima dos Santos
Paradigmas da Programação - Lista 6

```
import Data.Char
```

```
-- 1
```

```
calculaArea r = pi * r^2
```

```
-- 2
```

```
classificaTri x y z
```

```
  | x + y <= z || x + z <= y || y + z <= x = "NAOTRI" -- não é triângulo se: soma das  
medidas de dois segmentos menor que a medida do terceiro.
```

```
  | x == y && y == z = "Equilatero"                -- todos os lados iguais
```

```
  | x == y || y == z || x == z = "Isosceles"        -- dois lados iguais
```

```
  | otherwise = "Escaleno"                          -- todos os lados diferentes
```

```
-- 3
```

```
multiplica x y
```

```
  | y == 0 = 0
```

```
  | y > 0 = x + multiplica x (y - 1)
```

```
  | y < 0 = - (multiplica x (-y))
```

```
-- 4
```

```
multiplicaNaturais :: Int -> Int -> Int
```

```
multiplicaNaturais x y
```

```
  | y == 0 = 0
```

```
  | otherwise = x + multiplicaNaturais x (y - 1)
```

```
-- 5
```

```
menor x y z
```

```
  | x <= y && x <= z = x
```

```
  | y <= x && y <= z = y
```

```
  | otherwise = z
```

```
maior x y z
```

```
  | x >= y && x >= z = x
```

```
  | y >= x && y >= z = y
```

```
  | otherwise = z
```

```
-- 6
```

```
xor x y = (x || y) && not (x && y)
```

```

-- 7
clonaNumeros x = concatMap (\x -> [x, x]) x  -- \x -> [x, x] recebe um elemento x e
retorna uma lista com ele duplicado

-- 8
soma x = x!!0 + x!!1

-- 9
valorAbsoluto x = [0 .. abs x]

-- 10
nPar n = n `mod` 2 == 0
parOuImpar lista = map nPar lista

-- 11
soPar x = filter nPar x

-- 12
soMinusculas y = filter isLower y

-- 13
substituiVogais :: [Char] -> [Char]
vogalMin x = x `elem` "aeiou"
substituiVogais palavra = map (\x -> if vogalMin x then toUpper x else x) palavra

-- 14
acrescentaString lista = map (\x -> x ++ " Friboi") lista

-- 15
pertence _ [] = False  -- pra caso a lista esteja vazia
pertence x (y:ys)
    | x == y      = True
    | otherwise   = pertence x ys

-- 16
filtraLista [] = []
filtraLista (x:xs)
    | pertence x xs = filtraLista xs
    | otherwise    = x : filtraLista xs

-- 17

```

```
nPrimeiros n lista = take n lista
```

```
-- 18
```

```
multiplosTres = [x | x <- [0..300], x `mod` 3 == 0]
```